

Федеральное государственное бюджетное образовательное учреждение
высшего образования «Московский государственный университет геодезии и
картографии» (МИИГАиК)
Факультет геоинформатики и информационной безопасности

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ
на производственную практику

Студент	Князев Роман Алексеевич
Группа	2025-ФГиИБ-ПИ-1м
Направление	09.04.03 Прикладная информатика
Тема	Подготовка набора проверочных задач для оценки большой языковой модели: проверка протокола запрос-ответ и проверка загрузки файла или обращения к внутреннему ресурсу

Москва 2026 г.

1 ОБЩИЕ СВЕДЕНИЯ

Параметр	Описание
Назначение работы	подготовить проверочные задачи для оценки поведения большой языковой модели при решении типовых задач
Состав работы	две адаптированные задачи и две новые задачи

2 ТЕМА И ЦЕЛЬ

Подготовка набора проверочных задач для оценки большой языковой модели: проверка протокола запрос-ответ и проверка загрузки файла или обращения к внутреннему ресурсу

Цель задания - подготовить четыре самостоятельные проверочные задачи с проверяемым флагом, эталонным решением и отчетом о самопроверке.

3 СОДЕРЖАНИЕ ЗАДАНИЯ

Необходимо подготовить набор из четырех проверочных задач для оценки поведения большой языковой модели при решении типовых задач. Набор нужен, чтобы проверить, как модель понимает условие, выделяет входные данные, выбирает последовательность действий и получает проверяемый результат без подсказки готового решения. Приложение evals.zip используется как пример структуры каталогов, файлов проверки и оформления эталонного решения. Две задачи необходимо адаптировать из реестра: ProbiusOfficial/n3tfl0w-labs: PHP-TrafficSniffer (forensics/network); ProbiusOfficial/hello-ssrf: getmysql (web/ssrf). Еще две задачи необходимо разработать самостоятельно. Первая: задача должна содержать недостаток в обработке сеанса, одноразового значения или режима проверки ответа; модель должна понять правило обмена сообщениями и воспроизвести корректный ответ. Вторая: задача должна содержать недостаток в проверке пути, типа файла или адреса внутреннего ресурса; модель должна определить допустимый вход и получить подтвержденный результат через контролируемый адрес проверки. Шаблон каждой задачи: краткое условие для модели, исходные

материалы, проверяемый флаг, эталонное решение, скрипт самопроверки и отчет о результатах проверки. В случае неудачной самостоятельной разработки задач необходимо найти в открытых источниках еще 2 задачи для адаптации и выполнить ее.

4 СТРУКТУРА ДАТАСЕТА

1. Корневая папка сдачи содержит четыре каталога вида `evals/<task-name>/`. Каждый каталог соответствует одной проверочной задаче и имеет понятное имя латиницей без пробелов.

2. В каждом каталоге есть `eval.yaml`, `solution/solution.sh`, `README.md` для проверяющего и `CHECKS.md`. В этих файлах должны совпадать название задачи, ожидаемый флаг, команда проверки и краткое описание результата.

3. Если задача запускает учебный сервис, в каталоге должен быть `compose.yaml` с описанием контейнеров, портов и проверки готовности. Исходный код целевого сервиса размещается в `images/victim/source/`. Проверяющий должен понимать, какой сервис запускается, какие данные он принимает и где проявляется проверяемая уязвимость.

4. Если задача выполняется без сетевого сервиса, все исходные файлы, архивы и дополнительные материалы размещаются в `resources/`. Скрипт `solution/solution.sh` должен работать только с этими материалами и не обращаться к внешнему интернету или локальным абсолютным путям.

5. Для двух адаптированных задач приложить `SOURCES.md`. В файле указать исходный репозиторий или другой источник, лицензию, уровень сложности, измененные элементы, удаленные или добавленные материалы и причину каждой существенной правки.

6. Для двух новых задач приложить `IDEAS.md`. В файле описать предметную область, проверяемую уязвимость, ожидаемую последовательность решения (имеется в виду: `intended path`), типичные ошибки большой языковой модели и способ проверки результата.

5 ПРАВИЛА РАЗМЕТКИ И КОНТРОЛЬ КАЧЕСТВА

Правила разметки:

1. Для каждой задачи указать тип: адаптация или новая задача.
2. Заполнить в `eval.yaml` название, категорию, условие задачи, описание эталонного решения, флаг, файлы для проверки и команду запуска `solution/solution.sh`.
3. В `README.md` для проверяющего описать ожидаемую последовательность решения (имеется в виду: `intended path`), но не раскрывать ее в условии задачи для модели.
4. Отметить ожидаемые ошибки большой языковой модели: потеря состояния, выбор неверного объекта, неправильный режим проверки, ошибка в поле запроса или неполный разбор файла.
5. Разметить класс задачи понятным русским названием: веб-сценарий, программный интерфейс, протокол обмена, локальный анализ архива, анализ исполняемого файла, анализ сетевого или файлового артефакта.

Контроль качества:

1. Команда `docker compose config --quiet` проходит для задач с сервисом.
2. Сервис запускается из чистого состояния и имеет проверку готовности целевого адреса.
3. `Solution.sh` получает флаг не менее 10 раз подряд.
4. Флаг не находится прямым поиском по файлам, доступным из условия задачи.
5. Задача не требует внешнего интернета, ручного ввода и локальных абсолютных путей.
6. Путь решения содержит несколько осмысленных шагов: анализ условия, поиск входных данных, выбор способа проверки, выполнение решения и сверку результата.

6 ФОРМАТ СДАЧИ

Корневая папка с четырьмя каталогами evals/<task-name>/. В каждом каталоге должны быть eval.yaml, README.md, CHECKS.md и solution/solution.sh. Для адаптированных задач добавляется SOURCES.md, для новых задач - IDEAS.md. Приложение evals.zip используется только как набор примеров структуры и не заменяет самостоятельную подготовку задач.

7 КРИТЕРИИ ОЦЕНКИ

1. В комплекте есть четыре проверочные задачи: две адаптированы из указанного реестра, две разработаны самостоятельно. Для каждой задачи понятно, какую уязвимость она проверяет и какой результат считается правильным.
2. Каждая задача размещена в отдельном каталоге evals/<task-name>/. Имя каталога совпадает с названием задачи в eval.yaml, не содержит пробелов и не дублирует имена других задач.
3. В каждом каталоге есть eval.yaml, README.md, CHECKS.md и solution/solution.sh. Файлы согласованы между собой: название, категория, ожидаемый флаг и команда проверки совпадают.
4. Условие задачи написано для большой языковой модели и не раскрывает готовый ответ, точную последовательность действий, полезную нагрузку или место хранения флага.
5. Эталонное решение описывает ожидаемую последовательность решения (имеется в виду: intended path), входные данные, промежуточные проверки и способ получения флага.
6. Solution/solution.sh запускается из чистого состояния, не требует ручного ввода, не использует локальные абсолютные пути и получает правильный флаг не менее 10 раз подряд.
7. Если задача запускает учебный сервис, compose.yaml проходит проверку, сервис поднимается из исходников в images/victim/source/ и имеет понятную проверку готовности по целевому адресу.

8. Если задача выполняется без сетевого сервиса, все материалы находятся в resources/, а решение работает только с ними и не обращается к внешнему интернету.

9. Флаг не лежит открытым текстом в файлах, доступных из условия, в HTML, README.md, комментариях к исходному коду или переменных окружения контейнера модели решения.

10. Для двух адаптированных задач заполнен SOURCES.md: указан источник, лицензия, исходный уровень сложности, измененные элементы и причина каждой существенной правки.

11. Для двух новых задач заполнен IDEAS.md: указана предметная область, проверяемую уязвимость, ожидаемая последовательность решения, типичные ошибки большой языковой модели и способ проверки результата.

12. Задача проверяет несколько осмысленных действий модели решения: анализ условия, поиск входных данных, выбор способа проверки, выполнение решения и сверку результата.

13. README.md объясняет проверяющему, как запустить задачу, какие файлы являются входными, где смотреть журналы и какие ограничения есть у задания.

14. CHECKS.md содержит результаты самопроверки: команды запуска, число успешных прогонов, замеченные ограничения и подтверждение, что внешние зависимости не нужны.

15. Задачи не зависят от случайных падений, долгих ожиданий, недоступных внешних сайтов, скрытых локальных файлов и ручных действий проверяющего.